

Preliminaries of this Assignment

Released on April 1, 2026. Due at **23:59 on April 29, 2026**. You will have four weeks to complete this assignment. It is suggested to start working on this earlier.

In this assignment you will implement four robot learning algorithms that include advanced policy gradient method Proximal Policy Optimization (PPO), advanced value-based method Soft Actor Critic (SAC), model-based RL (DYNA), and Behavior Cloning (BC) with DAgger to replicate the teacher policy (trained PPO policy). The environments are simple (point-mass for goal reaching, and cart-pole for swinging up the inverted pendulum). The template code is provided as `hw3/` where all experiments can be run under it following the instructions in each questions. The code follows `hw2/` structure and same training procedure. Installation instructions are provided in `README.md` and no need to setup new python environment if you keep using the `env` from `hw2/`. All of the files that need to be changed in this assignment are located in the `scripts/` directory. No files outside of this directory should be modified. Locations where code needs to be modified are labeled `TODO`. No need to change `YAML` config files (i.e., no need to tune hyperparameters).

For your reference, the training time for PPO, SAC, DYNA, DAgger to have reasonable results are (on a regular laptop): **25 minutes, 3 hours, 1.5 hours, 1 minute**, respectively. *Please manage your time wisely.*

1 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is an on-policy policy gradient method that improves stability by restricting the magnitude of policy updates. Compared to vanilla policy gradient, PPO introduces a clipped objective that prevents excessively large updates to the policy.

The key idea is to define a probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t|\mathbf{s}_t)}, \quad (1)$$

and optimize the following clipped objective:

$$L(\theta) = \mathbb{E} [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]. \quad (2)$$

PPO also uses a learned value function $V(\mathbf{s})$ to reduce variance via advantage estimation.

Pseudo-code for PPO is shown in Algorithm 1.

ALGORITHM 1: Proximal Policy Optimization (PPO)

```

1:  $\theta \leftarrow$  initialize policy parameters
2:  $\phi \leftarrow$  initialize value function parameters

3: while not done do
4:   Collect trajectories using  $\pi_\theta$ 
5:   Compute returns and advantages

6:   for multiple epochs do
7:     for each minibatch do
8:       Compute ratio  $r_t(\theta)$ 
9:       Compute clipped objective
10:      Update actor parameters  $\theta$ 
11:      Update critic parameters  $\phi$ 
12:     end for
13:   end for
14: end while

15: return policy  $\pi_\theta$ 
```

To run PPO on the pointmass task, use the following command:

```
python run.py --mode train \
--env_config data/envs/dm_pointmass.yaml \
--agent_config scripts/dm_pointmass_ppo_agent.yaml \
--log_file output/pointmass_ppo_log.txt \
--out_model_file output/pointmass_ppo_model.pt \
--max_samples 1000000
```

Once a model has been trained, you can load a checkpoint and test the model with:

```
python run.py --mode test \
--env_config data/envs/dm_pointmass.yaml \
--agent_config scripts/dm_pointmass_ppo_agent.yaml \
--model_file output/pointmass_ppo_model.pt \
--visualize
```

1.1 Generalized Advantage Estimation

In `scripts/ppo_agent.py`, implement the `_calc_gae(rewards, values, dones)` method, which computes the advantage at each timestep using Generalized Advantage Estimation (GAE).

First compute:

$$\delta_t = r_t + \gamma V(\mathbf{s}_{t+1}) - V(\mathbf{s}_t), \quad (3)$$

then compute:

$$A_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}. \quad (4)$$

The output should be a tensor `adv` containing the advantage at each timestep.

then:

$$R_t = A_t + V(\mathbf{s}_t). \quad (5)$$

1.2 Critic Loss

In `scripts/ppo_agent.py`, implement the `_compute_critic_loss(values, returns)` method.

The critic loss is:

$$l(V) = \mathbb{E} [(V(\mathbf{s}) - R)^2]. \quad (6)$$

1.3 Actor Loss

In `scripts/ppo_agent.py`, implement the `_compute_actor_loss(logp, old_logp, adv)` method.

Compute:

$$r_t = \exp(\log \pi_{\theta} - \log \pi_{\theta_{\text{old}}}), \quad (7)$$

then:

$$l(\pi) = -\mathbb{E} [\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)]. \quad (8)$$

1.4 Deliverables

Train PPO policies for the pointmass task using the env config file `data/envs/dm.pointmass.yaml` and the corresponding agent config file `scripts/dm.pointmass_ppo_agent.yaml`. Train each model for at least 1M timesteps, this can be specified using the argument `--max_samples 1000000`. Tune the hyperparameters in the agent config file `scripts/dm.pointmass_ppo_agent.yaml`, so that the policy reliably reaches the goal and achieves stable performance. Submit the trained policy `pointmass_ppo_model.pt`. Plot a learning curve of the test return of each model using the plotting script `tools/plot_log/plot_log.py`.

2 Soft Actor-Critic (SAC)

Soft Actor-Critic (SAC) is an off-policy actor-critic algorithm that optimizes both expected return and entropy. By encouraging higher entropy, SAC improves exploration and stability in continuous control tasks.

The SAC objective is:

$$J(\pi) = \mathbb{E} [Q(\mathbf{s}, \mathbf{a}) - \alpha \log \pi(\mathbf{a}|\mathbf{s})]. \quad (9)$$

SAC uses two Q-functions to reduce overestimation bias, and target networks to stabilize learning.

Pseudo-code for SAC is shown in Algorithm 2.

ALGORITHM 2: Soft Actor-Critic

- 1: Initialize actor π_θ and critics Q_{ϕ_1}, Q_{ϕ_2}
 - 2: Initialize target networks
 - 3: Initialize replay buffer $\mathcal{D}$
 - 4: **while** not done **do**
 - 5: Collect transitions and store in $\mathcal{D}$
 - 6: **for** each update **do**
 - 7: Sample minibatch from $\mathcal{D}$
 - 8: Compute target y
 - 9: Update critics
 - 10: Update actor
 - 11: Update target networks
 - 12: **end for**
 - 13: **end while**
 - 14: return policy π_θ
-

To run SAC on the pointmass task:

```
python run.py --mode train \
--env_config data/envs/dm_pointmass.yaml \
--agent_config scripts/dm_pointmass_sac_agent_revised.yaml \
--log_file output/pointmass_sac_log.txt \
--out_model_file output/pointmass_sac_model.pt \
--max_samples 5000000
```

To test:

```
python run.py --mode test \
--env_config data/envs/dm_pointmass.yaml \
--agent_config scripts/dm_pointmass_sac_agent_revised.yaml \
--model_file output/pointmass_sac_model.pt \
--visualize
```

2.1 Action Sampling

In `learning/sac_model.py`, implement the `_sample_action(state)` method using the reparameterization trick (this is important to learn):

$$\mathbf{a} = \tanh(\mu + \sigma\epsilon), \quad \epsilon \sim \mathcal{N}(0, I). \quad (10)$$

2.2 Target Q Value and Critic Loss

In `scripts/sac_agent.py`, Implement `_compute_q_loss(...)`:

$$y = r + \gamma(1 - d) (\min(Q'_1, Q'_2) - \alpha \log \pi(a'|s')), \quad l(Q) = \mathbb{E} [(Q - y)^2]. \quad (11)$$

2.3 Actor Loss

Implement `_compute_actor_loss(...)`:

$$l(\pi) = \mathbb{E} [\alpha \log \pi(\mathbf{a}|\mathbf{s}) - \min(Q_1, Q_2)]. \quad (12)$$

2.4 Target Network Update

Implement `_soft_update_target_networks()`:

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta'. \quad (13)$$

2.5 Deliverables

Train SAC policies for the pointmass task using the env config file

`data/envs/dm_pointmass.yaml` and the agent config file

`scripts/dm_pointmass_sac_agent_revised.yaml`. Train each model for at least 5M timesteps using

`--max_samples 5000000`. Tune the hyperparameters so that the policy converges faster than PPO. Submit the trained policy

`pointmass_sac_model.pt`. Plot a learning curve using

`tools/plot_log/plot_log.py`.

3 Model-Based Reinforcement Learning (DYNA)

DYNA is a model-based reinforcement learning approach that combines real environment interaction with simulated experience from a learned dynamics model.

The model learns:

$$(\Delta \mathbf{s}, r) = f_{\psi}(\mathbf{s}, \mathbf{a}), \quad (14)$$

and predicts:

$$\mathbf{s}' = \mathbf{s} + \Delta \mathbf{s}. \quad (15)$$

Pseudo-code is shown in Algorithm 3.

ALGORITHM 3: DYNA

- 1: Initialize policy, critic, dynamics model
 - 2: Initialize replay buffers
 - 3: **while** not done **do**
 - 4: Collect real data
 - 5: Train dynamics model
 - 6: **for** model rollouts **do**
 - 7: Generate synthetic transitions
 - 8: **end for**
 - 9: Train RL using real + model data
 - 10: **end while**
 - 11: return policy
-

To run DYNA on cartpole:

```
python run.py --mode train \
--env_config data/envs/dm_cartpole_balance.yaml \
--agent_config scripts/dm_cartpole_balance_dyna_agent.yaml \
--log_file output/dyna_log.txt \
```

```
--out_model_file output/dyna_model.pt \  
--max_samples 500000
```

To test:

```
python run.py --mode test \  
--env_config data/envs/dm_cartpole_balance.yaml \  
--agent_config scripts/dm_cartpole_balance_dyna_agent.yaml \  
--model_file output/dyna_model.pt \  
--visualize
```

3.1 Dynamics Loss

Implement `_update_dynamics_model(...)`:

$$l = \|(s' - s) - \Delta s\|^2 + (r - \hat{r})^2 \quad (16)$$

3.2 Model Rollouts

Implement `_generate_model_rollouts()`:

- Roll out model
- Store synthetic transitions

3.3 Mix Training Batch

Implement `_sample_mixed_batch` to mix real replay and model replay, which is a key in DYNA.

3.4 Calculate Q and Actor Loss

Same as SAC.

3.5 Deliverables

Train DYNA agents for the cartpole balance task using the env config file

`data/envs/dm_cartpole_balance.yaml` and the agent config file

`scripts/dm_cartpole_balance_dyna_agent.yaml`. Train each model for at least 500k timesteps using

`--max_samples 500000`. Tune the dynamics model and rollout parameters to achieve stable balancing. Submit the trained policy `cartpole_dyna_model.pt`. Plot a learning curve using `tools/plot_log/plot_log.py`.

4 Behavior Cloning and DAgger

In this section, you will train a policy by imitating a PPO expert.

Behavior Cloning (BC) trains a policy via supervised learning:

$$l = \mathbb{E}[-\log \pi_{\theta}(\mathbf{a}_{\text{expert}}|\mathbf{s})]. \quad (17)$$

DAgger improves BC by aggregating data from states visited by the learned policy.

Pseudo-code is shown in Algorithm 4.

ALGORITHM 4: Behavior Cloning + DAgger

- 1: Initialize dataset using expert policy
 - 2: **while** not done **do**
 - 3: Rollout policy (teacher or student)
 - 4: Query expert for actions
 - 5: Aggregate dataset
 - 6: Train student using BC loss
 - 7: **end while**
 - 8: return policy
-

To run BC/DAgger:

```
python run.py --mode train \  
--env_config data/envs/dm_pointmass.yaml \  
--agent_config scripts/dm_pointmass_bc_dagger_agent.yaml \  
--log_file output/bc_dagger_log.txt \  
--out_model_file output/bc_dagger_model.pt \  
--max_samples 10000
```

To test:

```
python run.py --mode test \  
--env_config data/envs/dm_pointmass.yaml \  
--agent_config scripts/dm_pointmass_bc_dagger_agent.yaml \  
--model_file output/bc_dagger_model.pt \  
--visualize
```

4.1 Action Selection (Teacher vs Student)

In `scripts/bc_dagger_agent.py`, complete the `_decide_action(...)` method.

At each timestep, you must decide whether to use:

- the expert (teacher) action, or

- the student policy action.

The logic should follow:

- During early iterations (`self._iter < self._bc_pretrain_iters`), always use the teacher.
- After pretraining, use teacher action with probability `self._dagger_teacher_prob`.
- Otherwise, use the student policy.

This implements:

- Behavior Cloning (initial phase)
- DAgger (later phase with stochastic teacher forcing)

4.2 Behavior Cloning Loss

In `scripts/bc_dagger_agent.py`, implement the `_calc_bc_loss(...)` method.

The actor outputs a Gaussian policy $\pi_\theta(\mathbf{a}|\mathbf{s})$. The loss should be:

$$l = -\mathbb{E} [\log \pi_\theta(\mathbf{a}_{\text{expert}}|\mathbf{s})] - \beta \mathbb{E}[H(\pi_\theta)] \quad (18)$$

where:

- the first term is the negative log-likelihood (NLL),
- the second term is an entropy bonus (optional, controlled by `self._entropy_coef`).

Additionally, compute a mean squared error (MSE) metric between:

$$\|\pi(\mathbf{s}) - \mathbf{a}_{\text{expert}}\|^2 \quad (19)$$

for logging purposes (not required in the loss).

4.3 Deliverables

Train a BC+DAgger policy using the env config file `data/envs/dm_pointmass.yaml` and agent config file `scripts/dm_pointmass_bc_dagger_agent.yaml`. Use a pretrained PPO model as the teacher. Train for at least 10k timesteps using `--max_samples 10000`. Ensure that the DAgger policy improves over pure behavior cloning. Submit the trained policy `pointmass_bc_dagger_model.pt`. Plot a learning curve using `tools/plot_log/plot_log.py`.

5 Submission

Your submission should contain the following files:

- `ppo_agent.py`: code changes.
- `dm_pointmass_ppo_agent.yaml`: tuned hyperparameters for the pointmass task.
- `pointmass_ppo_model.pt`: trained model for the pointmass task.
- `pointmass_ppo_log.txt`: training log for the pointmass task.
- `pointmass_ppo_curve.png`: image of the learning curve for the pointmass task.

- `scripts/sac_agent.py`: code changes.
- `learning/sac_model.py`: code changes.
- `dm_pointmass_sac_agent_revised.yaml`: tuned hyperparameters for the pointmass task.
- `pointmass_sac_model.pt`: trained model for the pointmass task.
- `pointmass_sac_log.txt`: training log for the pointmass task.
- `pointmass_sac_curve.png`: image of the learning curve for the pointmass task.

- `dyna_agent.py`: code changes.
- `dm_cartpole_balance_dyna_agent.yaml`: tuned hyperparameters for the cartpole balance task.
- `cartpole_dyna_model.pt`: trained model for the cartpole balance task.
- `cartpole_dyna_log.txt`: training log for the cartpole balance task.
- `cartpole_dyna_curve.png`: image of the learning curve for the cartpole balance task.

- `bc_dagger_agent.py`: code changes.
- `dm_pointmass_bc_dagger_agent.yaml`: tuned hyperparameters for the BC+DAgger task.

- `pointmass_bc_dagger_model.pt`: trained model for the pointmass task.
- `pointmass_bc_dagger_log.txt`: training log for the BC+DAgger task.
- `pointmass_bc_dagger_curve.png`: image of the learning curve for the BC+DAgger task.

All files should be stored in a directory named `hw3/`, with two subdirectories `hw3/scripts/` for code and `hw3/output/` for the checkpoints, logs, and plots. Also, do not miss `learning/sac_model.py`. Then zip the directory for submission. Do not add any additional subdirectories.

You will also need to submit a short PDF report with required plots and answers. The single report should include:

- plots for each question, as required,
- others, if required, are specified in the individual questions.